

GPU Monitoring with DCGM

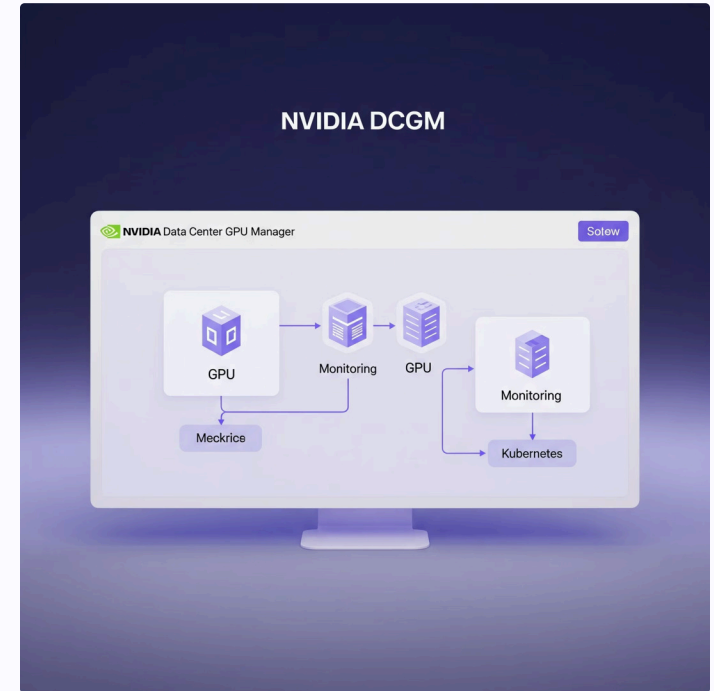
Deep visibility into NVIDIA GPU health and performance



What Is DCGM?

DCGM (Data Center GPU Manager) is NVIDIA's low-level toolkit for comprehensive GPU telemetry and management, providing:

- Detailed health, utilization, and error metrics
- Integration capabilities with monitoring stacks
- Seamless deployment in enterprise environments



Integrates natively with **Prometheus**, **Kubernetes**, **Slurm**, and DCGM Exporter

Why Use DCGM?

GPUs Are the Bottleneck

In AI infrastructure, GPUs represent the most expensive and constrained resource – making visibility critical

CPU Monitoring Insufficient

Traditional monitoring misses GPU-specific issues that can silently degrade performance

Comprehensive Detection

DCGM identifies utilization inefficiency, memory errors, thermal throttling, and process-level GPU usage

Without proper GPU monitoring, AI workloads can silently underperform or fail, wasting expensive compute resources

Key Metrics Exposed

Utilization

- GPU core percentage
- Memory utilization
- SM (Streaming Multiprocessor) occupancy

Memory

- Used vs. free memory
- Bandwidth throughput
- Memory controller utilization

Power & Thermals

- Wattage consumption
- Throttling events
- Fan speed and temperature

Errors & Process Stats

- ECC memory errors
- PCIe/NVLink connectivity issues
- PID-to-GPU resource mapping



DCGM Deployment Modes



Embedded Mode

Library integrated directly inside application with C and Python bindings for custom monitoring solutions



Standalone Mode

Command-line interface (dcgmi) for system administrators to perform on-demand diagnostics and checks



DCGM Exporter

Prometheus-compatible metrics endpoint for integration with existing monitoring infrastructure



Kubernetes Integration

Used by NVIDIA's Kubernetes device plugin for GPU management and health reporting

Each mode serves different operational needs, from development to production environments

CLI Examples with dcgmi

```
# List all GPUs on the system
```

dcgmi discovery

```
# Get utilization and memory usage for GPU 0
```

```
dcgmi stats --gpu 0
```

```
# Run short diagnostics (level 1)
```

```
dcgmi diag -r 1
```

```
# Enable health monitoring group
```

```
dcgmi health --set 1
```

The CLI provides quick checks, diagnostics, and automation capabilities for scripts and direct investigation

Pro Tip: Use these commands in your CI/CD pipelines to validate GPU health before deploying AI workloads

DCGM Exporter for Prometheus

DCGM Exporter bridges NVIDIA GPU metrics to your existing monitoring stack:

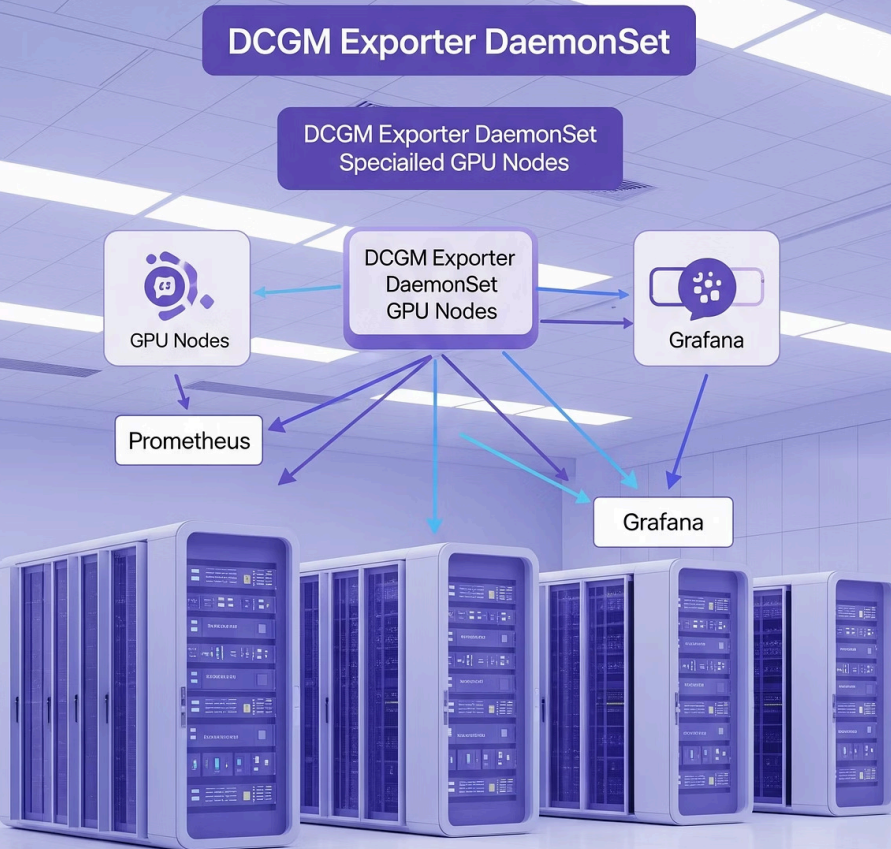
- Runs as a DaemonSet on every GPU node in Kubernetes
- Exposes `/metrics` endpoint for Prometheus scraping
- Provides standardized metrics with consistent naming

Example metrics exposed:

- `DCGM_FI_DEV_GPU_UTIL` - GPU utilization percentage
- `DCGM_FI_DEV_MEM_COPY_UTIL` - Memory controller activity
- `DCGM_FI_DEV_POWER_USAGE` - Power consumption in watts



Kubernetes Integration



NVIDIA GPU Operator

Includes DCGM Exporter by default, deploying it automatically on GPU nodes as a DaemonSet

Auto-Labeling

Metrics automatically labeled by node name, GPU index, and pod identifier for precise tracking

Scaling Integration

Combine with Horizontal Pod Autoscaler (HPA) or custom autoscalers for GPU-aware scaling decisions

This integration provides the foundation for GPU-aware cluster operations and workload scheduling

Health Monitoring Features

DCGM's health monitoring system provides proactive detection of GPU issues:

- Predefined health check groups for different monitoring scenarios
- Detection of XID errors (critical GPU exceptions)
- Monitoring of thermal thresholds and clock throttling events
- Integration with schedulers to **automatically quarantine failing GPUs**

This capability is essential for long-running AI training jobs that could otherwise waste days of compute time on faulty hardware



Visualization & Dashboards

NVIDIA provides ready-to-use Grafana dashboards for DCGM metrics:

- Per-GPU visualization of utilization, power, and temperature
- Cluster-wide views to identify underused or failing GPUs
- Custom alerting capabilities for temperature spikes and ECC errors
- Historical trend analysis for capacity planning



These dashboards provide at-a-glance visibility into your entire GPU fleet's health and performance

Best Practices

1 Deploy DCGM Exporter in All GPU Clusters

Make DCGM Exporter a standard component in your infrastructure to ensure consistent monitoring coverage

3 Implement Proactive Alerting

Configure alerts for ECC errors, overheating, and NVLink errors before they cause workload failures

2 Track Utilization Efficiency

Set up dashboards to identify underutilized GPUs – they represent expensive idle capacity that could be reallocated

4 Use Metrics for Capacity Planning

Leverage historical utilization data to make informed decisions about GPU investments and resource allocation

Key Takeaways



DCGM is the gold standard for GPU observability

Providing both node-level and cluster-level visibility through multiple deployment options

Enables proactive management

Through alerts, scaling, and health monitoring that prevents costly failures

Essential infrastructure for AI workloads

Making it possible to efficiently operate and optimize GPU resources at scale